

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.
4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.
5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

Answer:

Program :

```
def create_star_topology():
    n = int(input("Enter number of hosts: "))
    switch = "Central Switch"

    print("\nSTAR TOPOLOGY")
    print("-" * 40)

    lengths = []

    for i in range(1, n + 1):
        length = float(input(f"Enter Cat6 cable length for Host {i} (meters): "))
        lengths.append(length)

    print("\nTopology Representation:")
    print()

    for i in range(1, n + 1):
        print(f"Host {i} ----- {lengths[i-1]} m -----> {switch}")

    print("\nConnection Verification:")
    connected = True

    for i in range(1, n + 1):
        if lengths[i - 1] <= 0:
            print(f"Host {i}: Not Connected")
            connected = False
        else:
            print(f"Host {i}: Connected to {switch}")

    if connected:
        print("\nAll hosts are connected through the central switch.")
        print("Star topology provides reliable communication because")
        print("failure of one host cable does not affect other hosts.")
    else:
```

```
print("\nAll hosts are not properly connected.")
```

```
create_star_topology()
```

Output:

```
Enter number of hosts: 6

STAR TOPOLOGY
-----
Enter Cat6 cable length for Host 1 (meters): 10
Enter Cat6 cable length for Host 2 (meters): 13
Enter Cat6 cable length for Host 3 (meters): 14
Enter Cat6 cable length for Host 4 (meters): 15
Enter Cat6 cable length for Host 5 (meters): 11
Enter Cat6 cable length for Host 6 (meters): 21

Topology Representation:

Host 1 ----- 10.0 m -----> Central Switch
Host 2 ----- 13.0 m -----> Central Switch
Host 3 ----- 14.0 m -----> Central Switch
Host 4 ----- 15.0 m -----> Central Switch
Host 5 ----- 11.0 m -----> Central Switch
Host 6 ----- 21.0 m -----> Central Switch

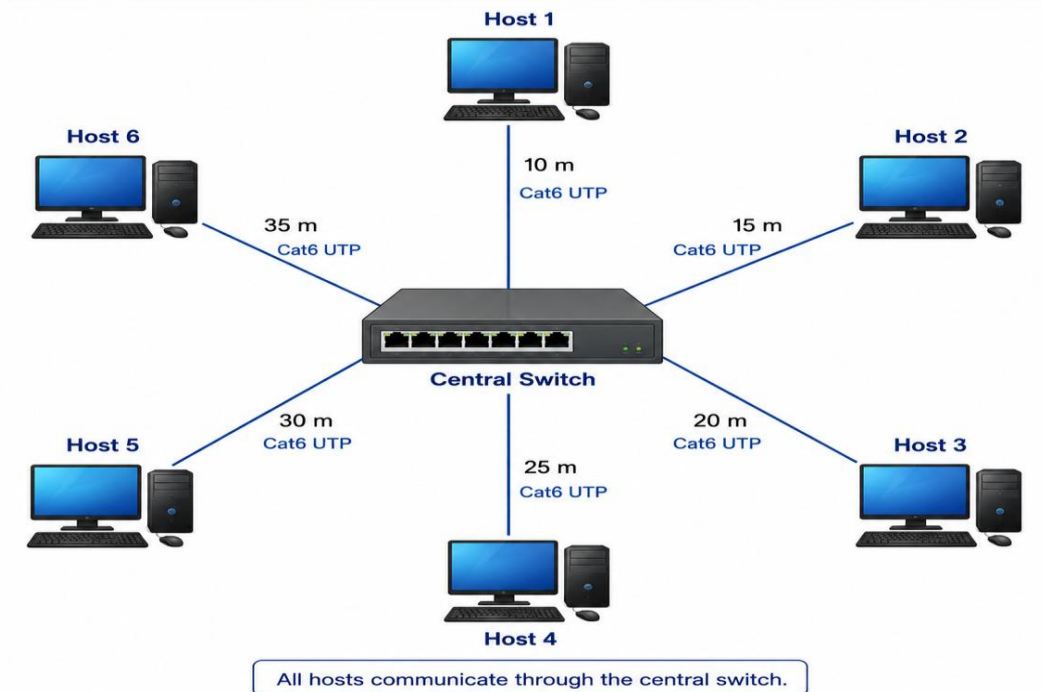
Connection Verification:
Host 1: Connected to Central Switch
Host 2: Connected to Central Switch
Host 3: Connected to Central Switch
Host 4: Connected to Central Switch
Host 5: Connected to Central Switch
Host 6: Connected to Central Switch

All hosts are connected through the central switch.
Star topology provides reliable communication because
failure of one host cable does not affect other hosts.

...Program finished with exit code 0
Press ENTER to exit console.
```

Graphical representation

Star Topology – Centralized LAN Network



Description:

The first image presents a clean, professional **star topology network diagram** designed in the style of a college networking laboratory illustration. The setting is an abstract, white digital workspace with no physical room or environmental distractions. At the exact center sits a modern Ethernet switch labeled “**Central Switch,**” serving as the visual and functional focal point. Six desktop computer icons—Host 1 through Host 6—are arranged evenly around it in a circular formation, with individual Cat6 UTP cables extending directly from each host to the switch. The design uses a restrained palette of white, cool blue, and gray, creating a precise academic and technological mood. Straight cable connections act as strong leading lines toward the center. Sharp vector icons, uniform line weights, clear labels, and balanced spacing give the composition a polished Cisco-style laboratory appearance.

2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

Answer

Program :

```
def validate_segment(length_m):  
    if length_m <= 100:  
        return True  
    else:  
        return False, "Error: Cable length exceeds the maximum limit of 100  
meters."
```

```
test_cases = [50, 100, 120]
```

```
for length in test_cases:  
    result = validate_segment(length)  
  
    print(f"Cable Length: {length} meters")  
  
    if result is True:  
        print("Result: Valid cable segment")  
    else:  
        print("Result:", result[1])  
  
    print()
```

output:

```

Cable Length: 50 meters
Result: Valid cable segment

Cable Length: 100 meters
Result: Valid cable segment

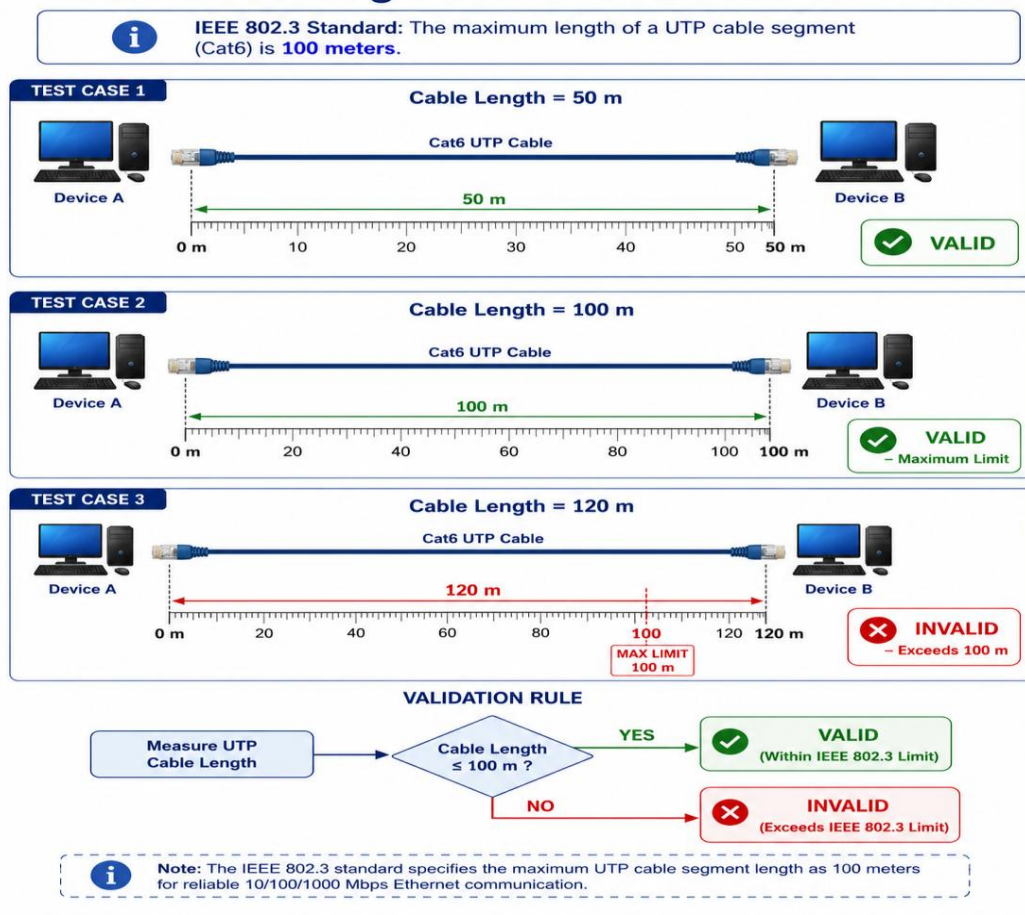
Cable Length: 120 meters
Result: Error: Cable length exceeds the maximum limit of 100 meters.

...Program finished with exit code 0
Press ENTER to exit console.

```

Graphical representation:

UTP Cable Segment Validation – IEEE 802.3



Description:

The second image is a **technical infographic-style network illustration** focused on validating UTP Ethernet cable length. Against a bright white background, three separate horizontal cable-testing scenarios are presented between professional Ethernet endpoint icons. Each cable is accompanied by a ruler-like measurement scale, clearly representing **50 m, 100 m, and 120 m** test cases. Blue and gray dominate the visual palette, while contrasting status indicators distinguish valid and invalid results, giving the diagram an analytical and instructional character. The 100-meter example receives particular visual emphasis as the maximum permissible segment length, while the 120-meter connection is visually marked as exceeding the limit. Clean geometric shapes, straight cable lines, consistent typography, and carefully aligned panels create a structured laboratory-record aesthetic. A compact decision flow at the bottom reinforces the relationship between cable length and validation status.

Thematic relationship: Both images form a connected networking laboratory series: the first demonstrates **network topology and connectivity**, while the second focuses on **physical Ethernet cable-length validation**.